

Karanbir Singh Pelia

📍 Stony Brook, NY 📞 +1 (631) 710-5508 ✉ karanbirsinghpelia@gmail.com in karanbir-pelia 🌐 karanbir-pelia 📧 karanbir-pelia

Education

Stony Brook University | MS, Computer Science

Stony Brook, NY | **Aug 2024 – May 2026**

Coursework: Data Science, HCI, Computer Graphics, Advanced Algorithms, Distributed Systems, Data Management

Savitribai Phule Pune University | BE, Computer Engineering

Pune, MH | **Sep 2020 – Jun 2024**

Coursework: Data Structures and Algorithms, Operating Systems, Computer Networks, Database Systems, Machine Learning

Experience

Infrd Inc. | Software Developer Intern

San Jose, CA | **May 2025 – Jan 2026**

Technologies: Python, Jira, Confluence, Grafana, Jenkins, Git, GitHub, REST APIs, FastAPI, HTML, CSS, JavaScript, MongoDB, Postman

- Built an internal evaluation platform with FastAPI and REST APIs, enabling automated accuracy benchmarking across 6+ document processing pipelines for 100+ document types.
- Automated test pipelines for classification and extraction services, improving accuracy tracking, reducing manual QA effort by over 50%.
- Delivered 5+ document processing backend features across Agile sprints and enhanced existing workflows based on sprint feedback.
- Diagnosed and resolved 3+ production issues in classification backend workflows using systematic Grafana log analysis.
- Integrated AI coding assistants (Copilot, Claude Code) into sprint workflows, meaningfully accelerating feature delivery while enforcing code review standards for correctness and edge case coverage.

Projects

MediGrid | Fault-Tolerant Medical Resource Availability Platform

Feb 2026 – Apr 2026

Technologies: Java, Spring Boot, Kafka, gRPC, RAFT, PostgreSQL, Redis, Docker, Kubernetes

- Engineered a fault-tolerant distributed medical resource platform using Spring Boot and RAFT over gRPC, maintaining write availability with majority quorum under node failures.
- Built a real-time REST → Kafka → RAFT → PostgreSQL ingestion pipeline for resource updates with zero-availability alerts.
- Implemented Redis cache-aside queries with 30-second TTLs and PostgreSQL fallback, improving lookup speed while preserving reliability during cache failures.
- Validated a 5-node cluster end-to-end — leader election, write consistency, critical alerts, and Redis cache propagation — with a scripted failure-injection suite simulating 2-node simultaneous crashes.

Site to Slides | Webpage-to-Presentation Automation Service

Mar 2025 – Apr 2025

Technologies: Python, Firecrawl API, REST APIs, WebSockets, JWT Auth, Web Scraping, Browser DevTools

- Engineered a 3-stage automation pipeline (scrape → authenticate → generate) that converts any public URL into a shareable AI presentation by orchestrating 5 REST endpoints and 1 WebSocket stream across an undocumented third-party API.
- Reverse-engineered 8 undocumented endpoints via browser network inspection, implementing a 2-layer JWT auth flow (password → access token → refresh token) with a 5-minute expiry buffer and automatic fallback for session stability.
- Built an LLM-powered HTML extraction layer using Firecrawl that parses raw webpage content into 4–6 structured sections with per-section image mapping, feeding a retry-backed slide generator (up to 4 attempts per slide) to ensure reliable output.

Fatigue Sense | Real-Time Driver Fatigue Monitor

Oct 2024 – Nov 2024

Technologies: Python, TensorFlow/Keras, OpenCV, MediaPipe Pose/Hands, NumPy, Pygame, scikit-learn, Matplotlib

- Built a real-time driver safety system unifying a Keras CNN with MediaPipe Pose and Hand tracking across 3 concurrent detection pipelines, flagging fatigue, posture, and hand-position risks with 95% classification accuracy.
- Engineered a multi-modal inference pipeline tracking 21 hand landmarks and 4 distinct posture conditions per frame, with time-gated alert logic (2–3s sustained thresholds) to suppress false positives and deliver sub-second visual and auditory warnings.

AgroDoc | End-to-End Plant Disease Detection App

Sep 2023 – Mar 2024

Technologies: Python, TensorFlow/Keras, Streamlit, NumPy, OpenCV, Matplotlib, Jupyter Notebook, Kaggle

- Built a full-stack plant disease detection web app using a TensorFlow CNN (modular preprocessing → inference → rendering pipeline) served via Streamlit, classifying 38 conditions across 14 crops at 97.38% accuracy on 87,000+ images.
- Improved model generalization by applying data augmentation and learning rate scheduling, recovering ~2–3 percentage points of accuracy over a naive baseline and validating gains on a held-out test split.

Publications

Advances in Plant Disease Detection and Classification Systems

Scrivener Publishing LLC | **Nov 2025**

Co-authored a chapter published in **Optimizing AI Applications for Sustainable Agriculture**, documenting findings from my BE final-year research project **AgroDoc**. Covers deep learning approaches — CNNs, transfer learning, and data augmentation — for plant disease detection, with discussion of IoT integration and precision agriculture applications.

Technical Skills

Languages & Libraries: Python, Java, JavaScript, C++, SQL, HTML, CSS, TensorFlow, OpenCV, NumPy

Frameworks & Backend: Spring Boot, FastAPI, Kafka, gRPC, REST APIs, Micrometer, JUnit 5, Mockito, Pytest

Databases & DevOps: PostgreSQL, MongoDB, MySQL, Redis, Docker, Kubernetes, Git, GitHub, Jenkins, CI/CD

Observability & Tools: Prometheus, Grafana, Postman, Jira, Confluence, Agile, OOP Design Patterns, Scalable System Design

AI & Agentic Development: Claude Code, GitHub Copilot, Cursor, Codex, Google Antigravity, Ollama, Prompt Engineering, LLM Output Evaluation